

Árvores Balanceadas: AVL

Parte 2

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Análise Inclusão

- Suponha nó q foi incluído em T
- Se após inclusão todos nós se mantiveram **balanceados** então
 - A árvore manteve-se **AVL** e não há nada a efetuar
- Caso contrário
 - Seja p o nó mais próximo às folhas de T que se tornou **desbalanceado**
 - Não há ambiguidade na escolha de p , pois qualquer subárvore de T que se tornou **desbalanceada** após a inclusão de q deve necessariamente conter p
 - Então p se encontra no caminho de q à raiz de T , e sua escolha é única

T Desbalanceada

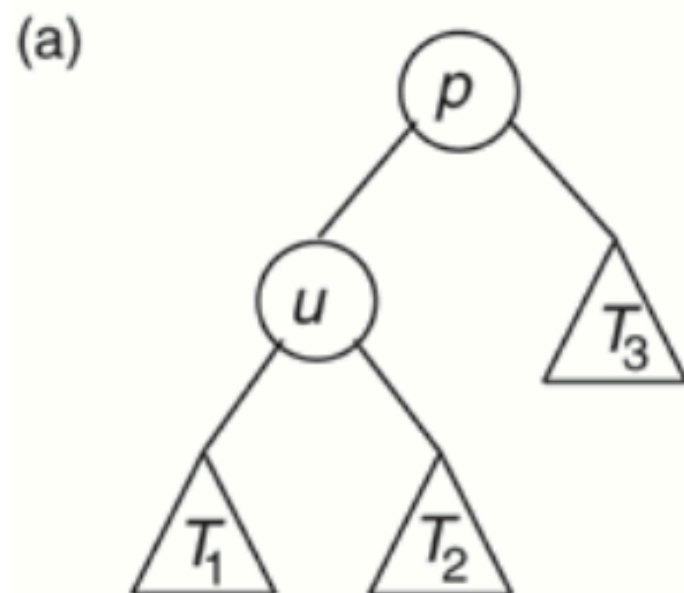
- Sejam:
 - $h_E(p)$ altura da subárvore esquerda
 - $h_D(p)$ altura da subárvore direita
- Como T está **desbalanceada**, $|h_E(p) - h_D(p)| > 1$
- Então, conclui-se que $|h_E(p) - h_D(p)| = 2$, pois:
 - $\Rightarrow T$ era uma árvore **AVL**
 - $\Rightarrow$ a inclusão de um nó não pode aumentar em mais de uma unidade a altura de qualquer subárvore
- Podem-se identificar vários casos

Caso 1: $h_E(p) > h_D(p)$

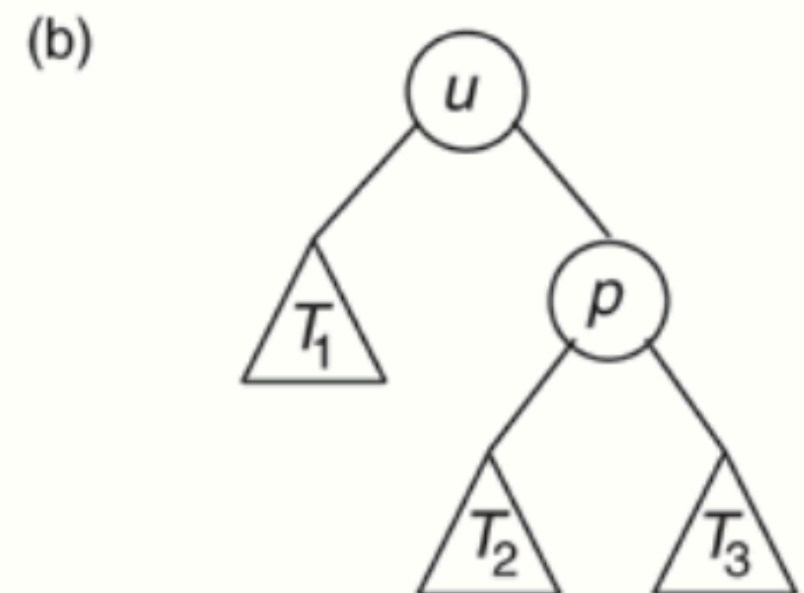
- Então, q pertence à subárvore esquerda de p . Além disso, p possui o filho esquerdo $u \neq q$. Pois, caso contrário, p não estaria **desbalanceado**. Finalmente, por esse mesmo motivo, sabe-se que $h_E(u) = h_D(u)$.

Caso 1.1: $h_E(u) > h_D(u)$

- Essa situação corresponde à da Figura (a), sendo q um nó pertencente a T_1 . Observe que $h(T_1) - h(T_2) = 1$ e $h(T_2) = h(T_3)$. Consequentemente, a aplicação da rotação direita da raiz p transforma a subárvore considerada na da Figura (b), o que restabelece a regulagem de p .



→
rotação
direita

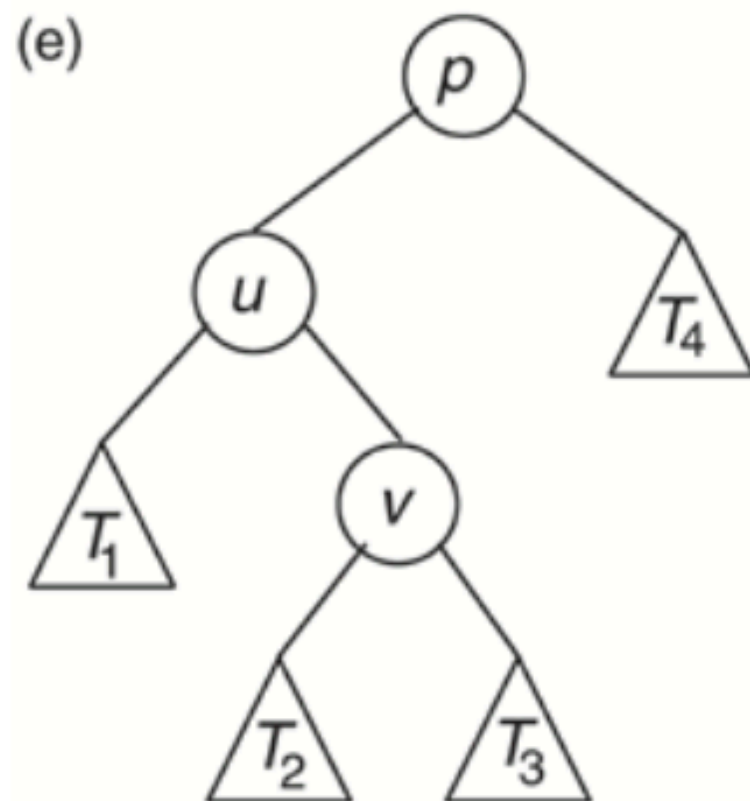


Caso 1.2: $h_D(u) > h_E(u)$

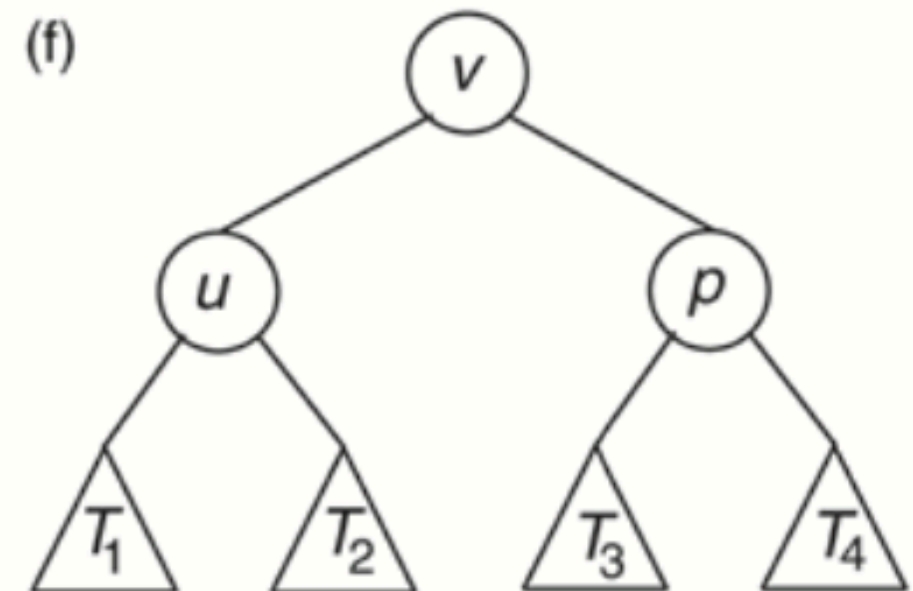
- Então u possui o filho direito v , e a situação é ilustrada na Figura (e). Nesse caso, vale

$$|h(T_2) - h(T_3)| \leq 1 \text{ e} \\ \max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$$

- Aplica-se então a rotação dupla direita da raiz p



→
rotação
dupla
direita



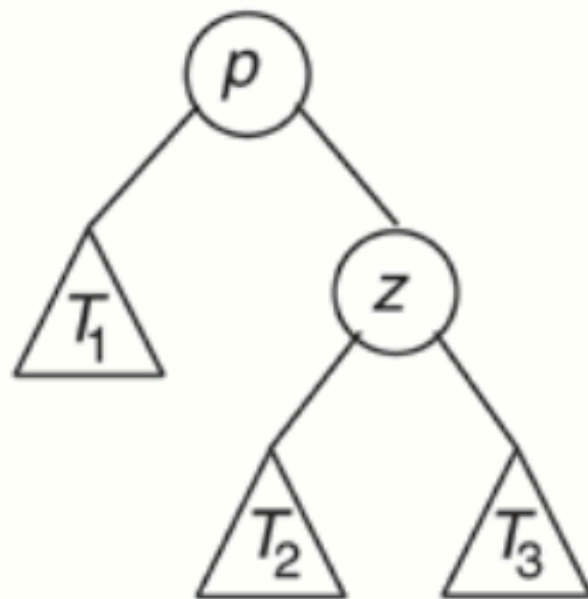
Caso 2: $h_D(p) > h_E(p)$

- Análogo ao caso 1.

Caso 2.1: $h_D(z) > h_E(z)$

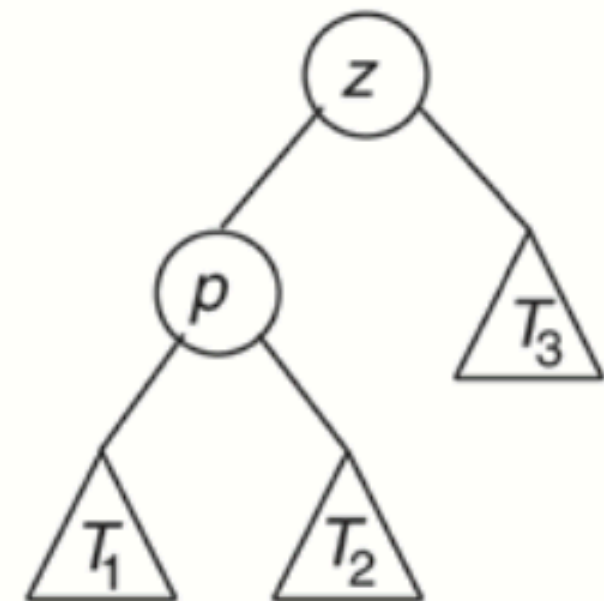
- Análogo ao caso 1.1.

(c)



→
rotação
esquerda

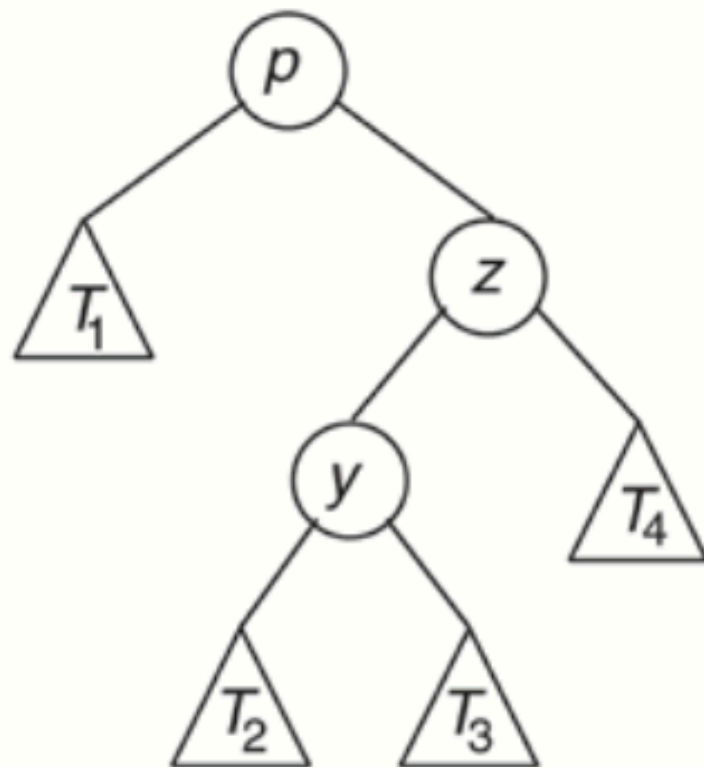
(d)



Caso 2.2: $h_E(z) > h_D(z)$

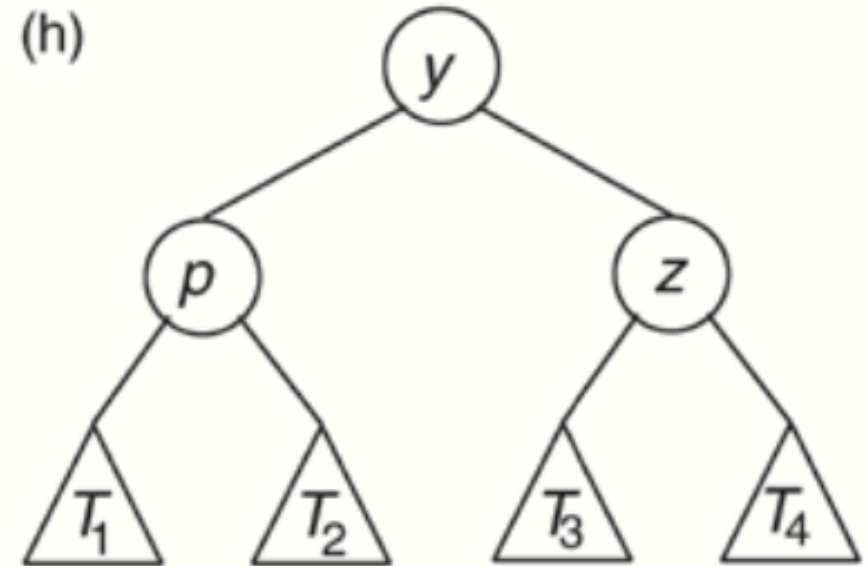
- Análogo ao caso 1.2.

(g)



→
rotação
dupla
esquerda

(h)



Regulagem

- O **balanceamento** de p é restabelecida pela aplicação apropriada de uma das **transformações**
 - ⇒ Rotação direita (LL)
 - ⇒ Rotação esquerda (RR)
 - ⇒ Rotação dupla direita (LR)
 - ⇒ Rotação dupla esquerda (RL)
- O pai de p , após a transformação, encontra-se também **balanceado**, pois a transformação diminui de uma unidade à altura da subárvore da raiz p
- Consequentemente, uma única transformação é suficiente para **balancear** T

Figura: Regulagem da árvore AVL (a)

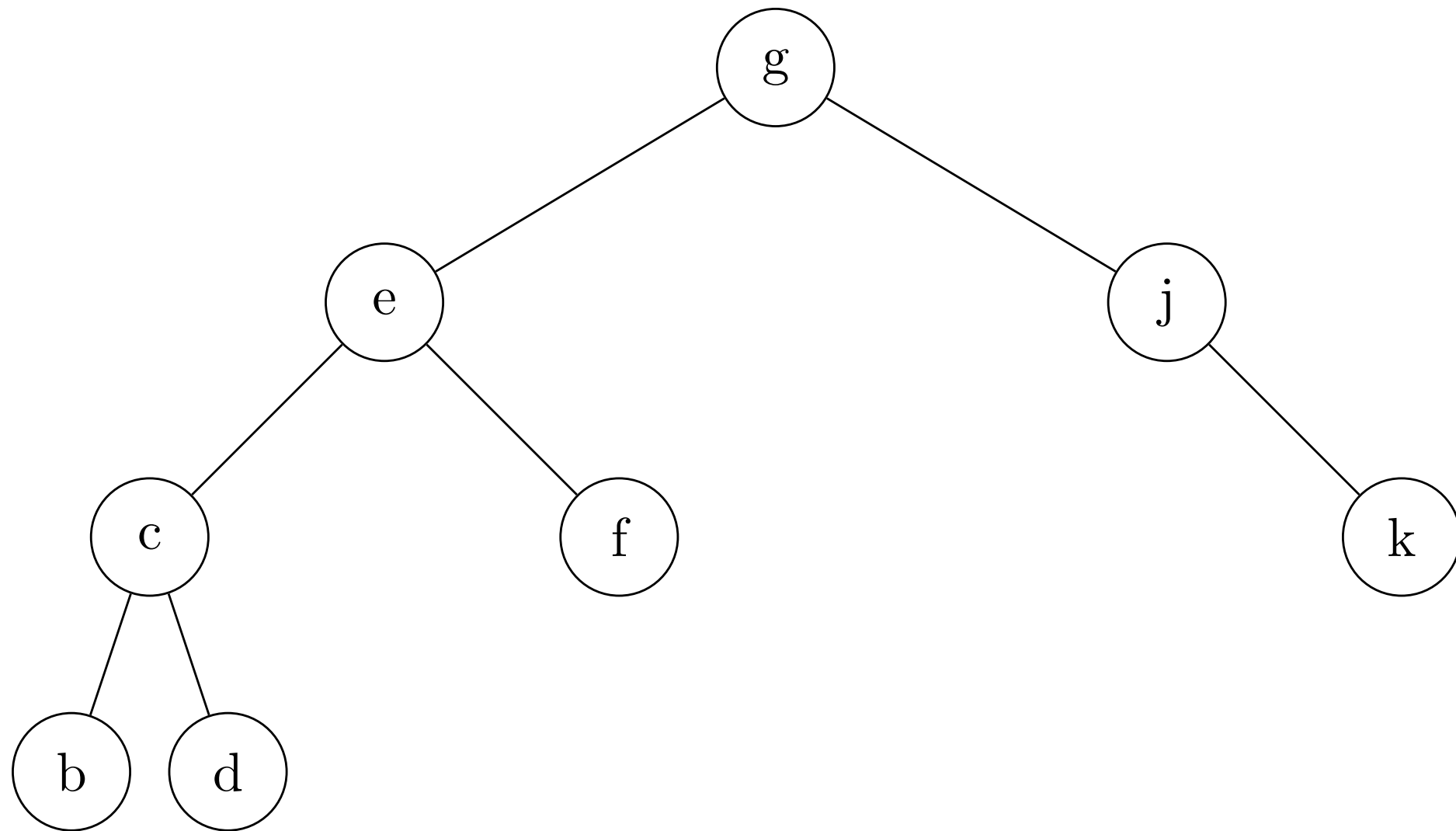


Figura: Regulagem da árvore AVL (b)

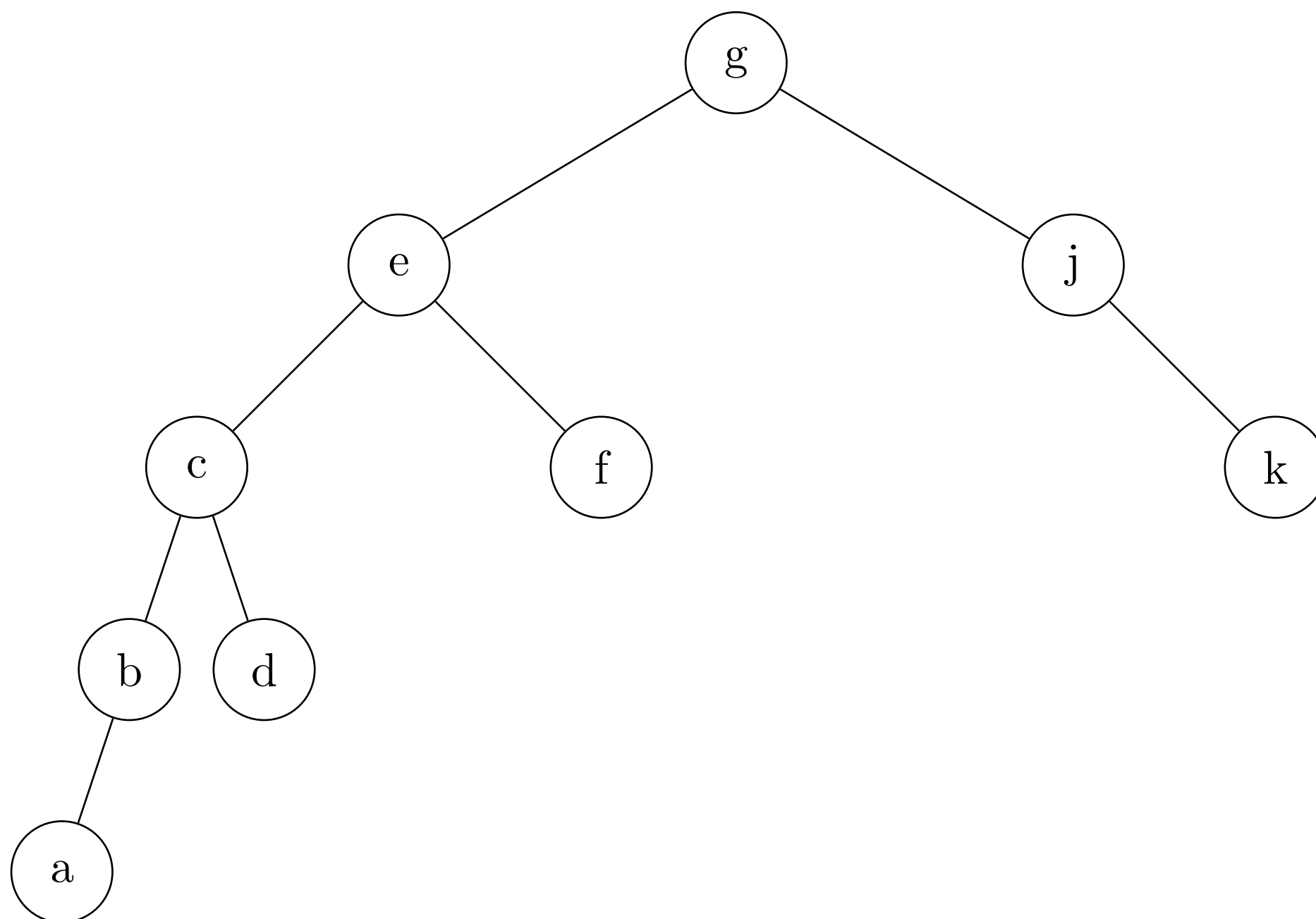
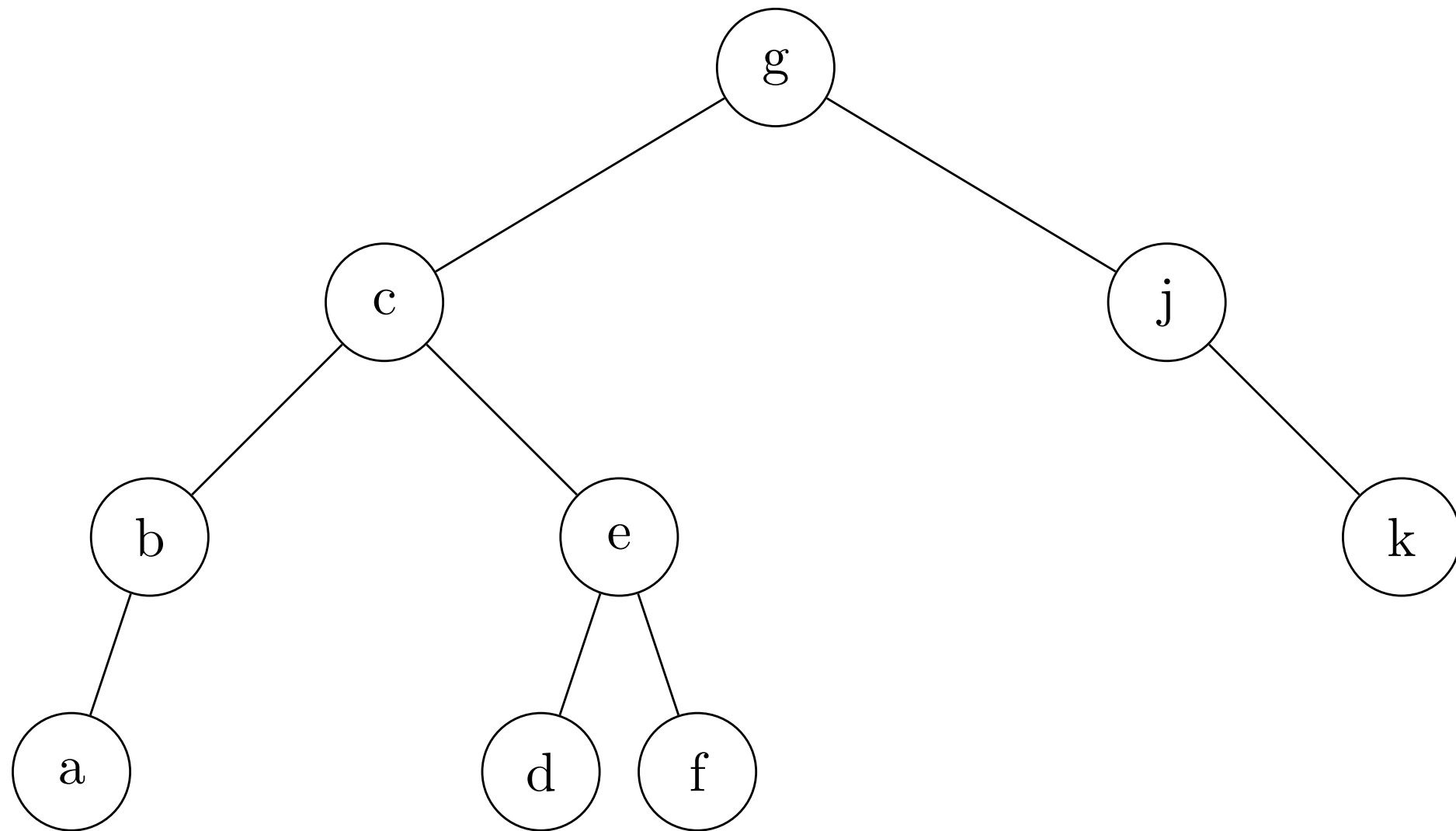


Figura: Regulagem da árvore AVL (c)



Implementação da Inclusão

- Seja T uma **árvore AVL** e x a **chave** a ser incluída em algum novo nó q
- Inclusão pode ser dividida em duas etapas

Implementação da Inclusão: Etapa 1

- **Etapa 1: Busca** em T por x
 - Se x já se encontra em T :
⇒ o processo deve ser encerrado
 - Caso contrário:
 - **Busca** encontra o local correto do novo nó. Segue-se, então, a inclusão propriamente dita
 - Se operação não tornou nenhum **nó desbalanceado**:
⇒ o processo termina, pois T permanece **AVL**
 - Caso contrário:
⇒ regulagem de T deve ser efetuada

Implementação da Inclusão: Etapa 2

- **Etapa 2:** A última etapa consiste no reconhecimento da **operação de rotação** indicada ao caso e em sua posterior execução
 - Isto devolve a T a condição de **árvore AVL**

Verificar Nó Desbalanceado

- Como verificar se algum nó v de T se tornou **desbalanceado** após a inclusão??

Verificar Nó Desbalanceado:

Alternativa 1

- Determinar alturas das duas subárvores e subtrai
- Essa operação pode ser realizada percorrendo-se a subárvore de raiz v em T
 - Consome $O(|T(v)|)$, ou seja, $O(n)$
- Qualquer nó do caminho de q até a raiz da árvore pode ter se tornado **desbalanceado**
- Aplicar esse procedimento leva a um algoritmo de complexidade $O(n \log n)$ (fora de cogitação)

Verificar Nó Desbalanceado: Alternativa 2

- Percorrer T de baixo para cima e armazenando-se o valor da altura de cada subárvore
 - Complexidade $O(n)$
- Naturalmente, $h(v) = 1 + \max\{h_E(v), h_D(v)\}$
- Ainda não é $O(\log n)$!!

Verificar Nó Desbalanceado:

Alternativa 3

- Define-se o rótulo **balanço(v)**, para cada nó v de T como $bal(v) = h_D(v) - h_E(v)$
- v está **balanceado** se e somente se $-1 \leq bal(v) \leq 1$
- Como atualizar $bal(v)$ de forma eficiente, tendo em vista a inclusão de q ??
- Complexidade $O(\log n)$

Verificar Nó Desbalanceado: Alternativa 3 (2)

- Se $q \in T_{esq}(v)$ e essa inclusão aumentar altura dessa subárvore, então
 - Subtrai-se uma unidade de $bal(v)$
 - Se valor resultar em -2 , v se tornará **desbalanceado**
- Se $q \in T_{dir}(v)$ e essa inclusão aumentar altura dessa subárvore, então
 - Adiciona-se uma unidade de $bal(v)$
 - Se valor resultar em 2 , v se tornará **desbalanceado**

Figura: Balanço da Árvore AVL (a)

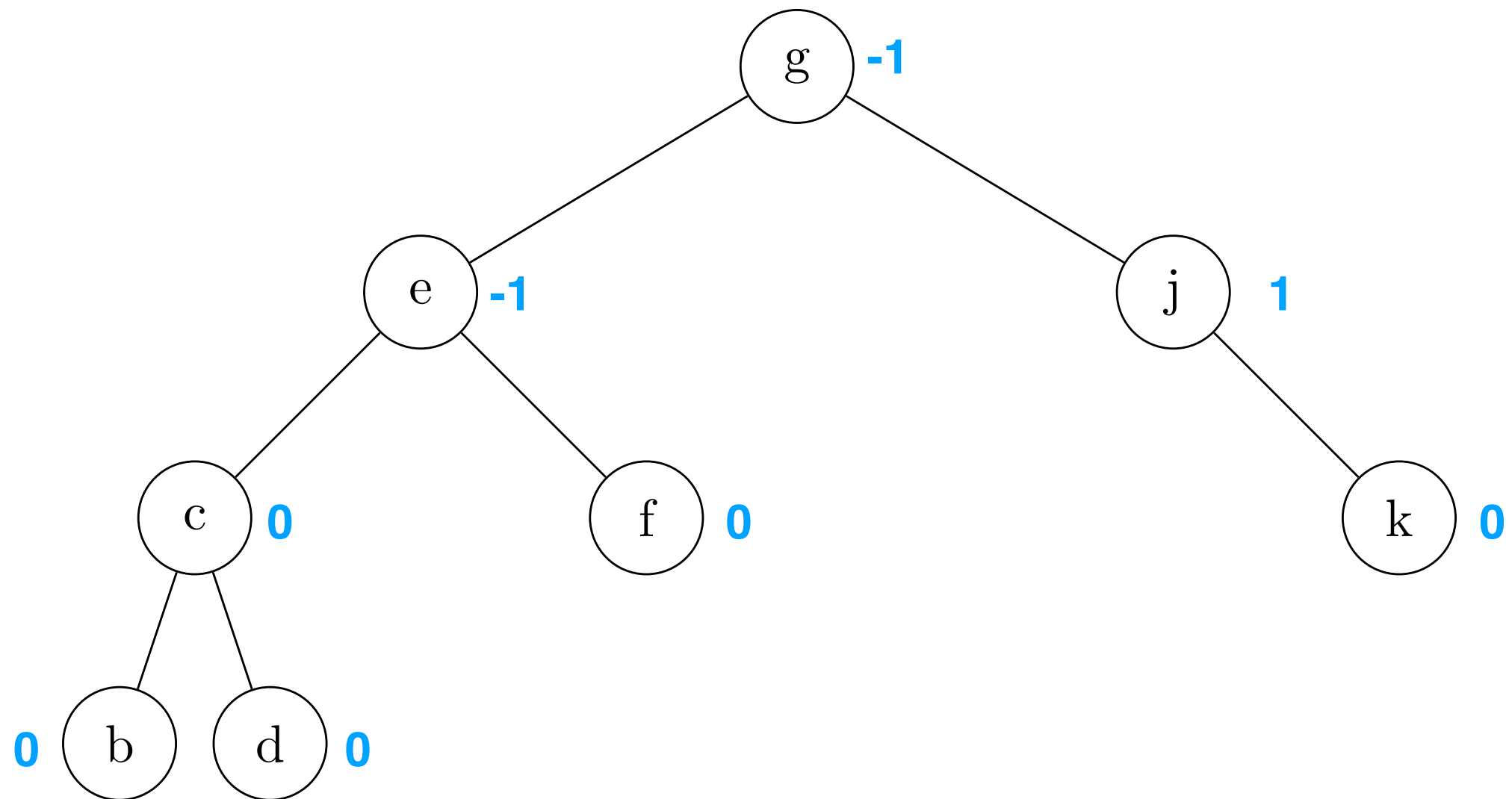


Figura: Balanço da Árvore AVL (b)

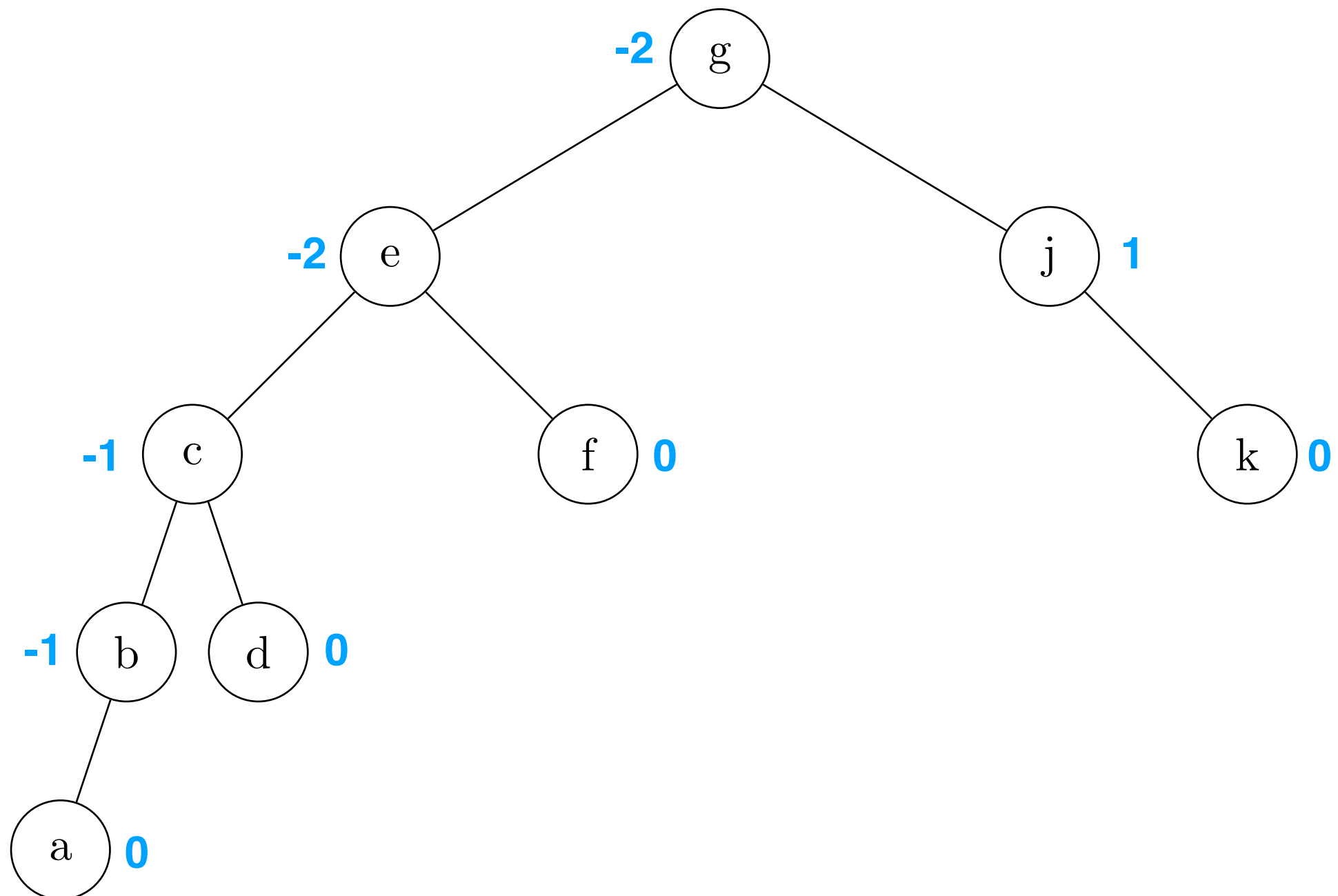
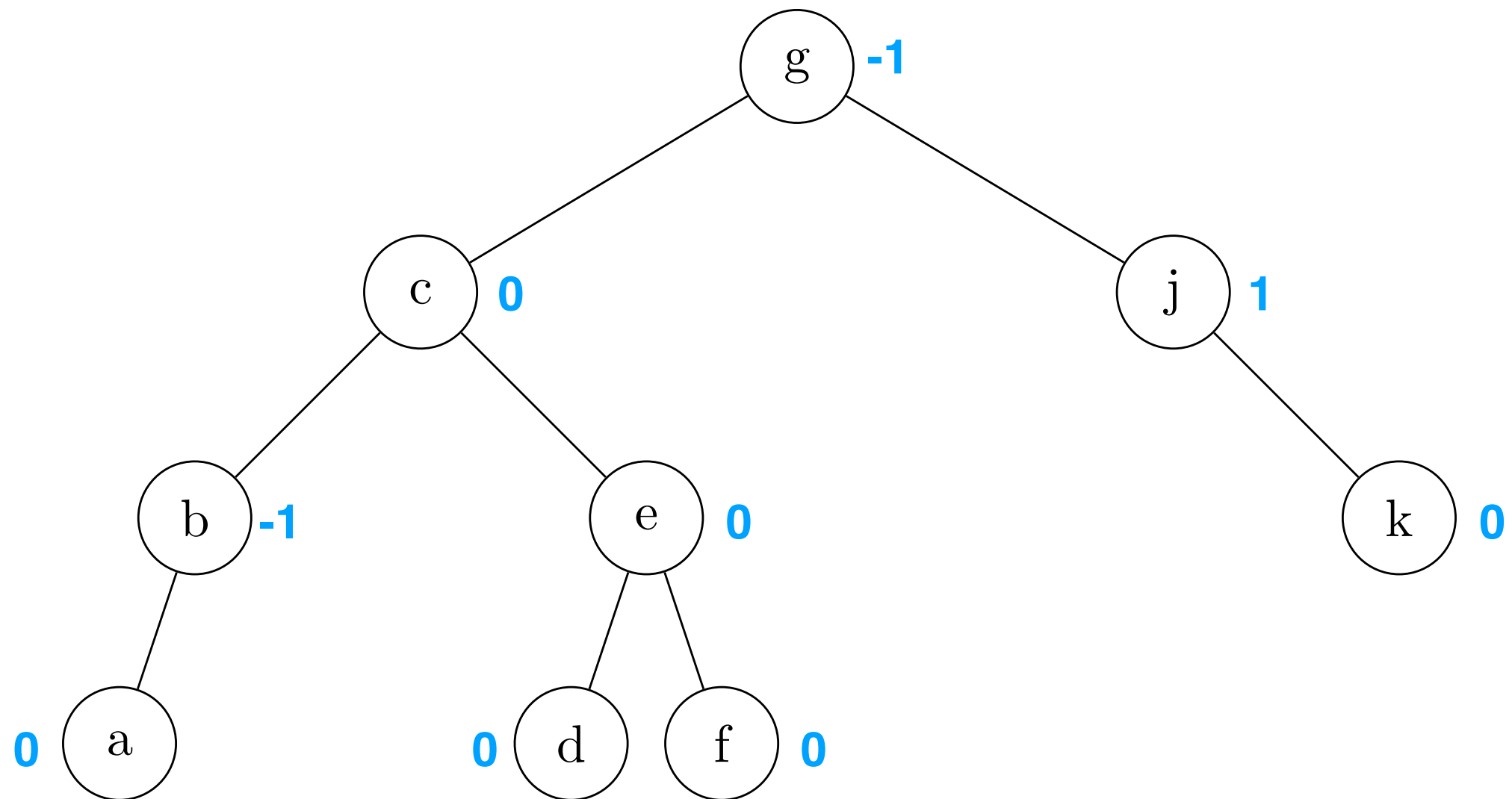


Figura: Balanço da Árvore AVL (c)



Verificar Nó Desbalanceado: Alternativa 3 (3)

- Em que casos a inclusão de q provoca um **acréscimo** na altura $h(v)$ da subárvore $T(v)$??
 - Inserção de q altera altura da subárvore esquerda ou direita de seu nó pai w
 - A situação de **balanço** permite avaliar se essa alteração pode, ou não, se propagar aos outros nós v do caminho de w até a raiz da árvore

Verificar Nó Desbalanceado: Alternativa 3 (4)

- Suponha q é inserido na **subárvore esquerda** de v
- Análise inicia com $v = w$ e prossegue em **nós ancestrais**, de forma recursiva
- Processo encerra quando é constatado que altura de $T(v)$ não foi modificada, ou de que v se tornou **balanceado**
- Três casos distintos podem ocorrer

Caso 1: $bal(v) = 1$ antes inclusão

- Nesse caso, $bal(v)$ se torna 0
 - Altura de $T(v)$ não foi modificada
- Consequentemente, as alturas dos nós restantes do caminho até a raiz da árvore não se alteram

Caso 2: $bal(v) = 0$ antes inclusão

- $bal(v)$ se torna -1
 - Altura de $T(v)$ foi modificada
- Consequentemente, os nós restantes do caminho até a raiz também podem ter suas alturas modificadas e devem ser analisados
- Se v é a raiz de T :
 - Análise se encerra, pois nenhum nó se tornou **desbalanceado**
- Caso contrário:
 - Repetir processo com v substituído por seu pai

Caso 3: $bal(v) = -1$ antes inclusão

- $bal(v)$ se torna -2 e o nó está **desbalanceado**
- **Rotação** correta deve ser empregada
 - Qualquer rotação implica que a subárvore resultante tenha a mesma altura da subárvore antes da inclusão
 - Alturas dos **ancestrais** de v não mais necessitam de avaliação

Inserção na Subárvore Direita

- Para uma inserção na subárvore direita de v , casos simétricos devem ser considerados

Algoritmo: Insere AVL

```
1. procedimento ins-AVL(x, pt, h)
2.   se pt = NULO então
3.     pt := novo_no(x)
4.     h := V
5.   senão
6.     se x = pt->chave então
7.       pare
8.     se x < pt->chave então
9.       ins-AVL(x, pt->esq, h)
10.      se h então
11.        caso pt->bal seja
12.          1: pt->bal := 0; h := F
13.          0: pt->bal := -1
14.          -1: caso1(pt, h) % rebalanceamento
15.      senão
16.        ins-AVL(x, pt->dir, h)
17.      se h então
18.        caso pt->bal seja:
19.          -1: pt->bal := 0; h := F
20.          0: pt->bal := 1
21.          1: caso2(pt, h) % rebalanceamento
22.
```

Inserir AVL

- Apenas uma **rotação**, **simples** ou **dupla**, é necessária
- Procedimento *insira_AVL* produz o efeito de percorrer a árvore desde a raiz até o nó q incluído
- Complexidade é $O(\log n)$

Operação Exclusão

- **Exclusão** também pode ser efetuada em $O(\log n)$ passos
- Após **exclusão** do nó desejado, a ideia consiste em verificar se a árvore se tornou **desbalanceada**
- Computação pode se restringir ao exame dos nós no caminho da raiz até alguma folha
- Contudo, não basta necessariamente uma única rotação para **balancear** a árvore!!
 - Número de operações de rotação necessárias pode atingir $O(\log n)$

Figura: Remoção em Árvore AVL (a)

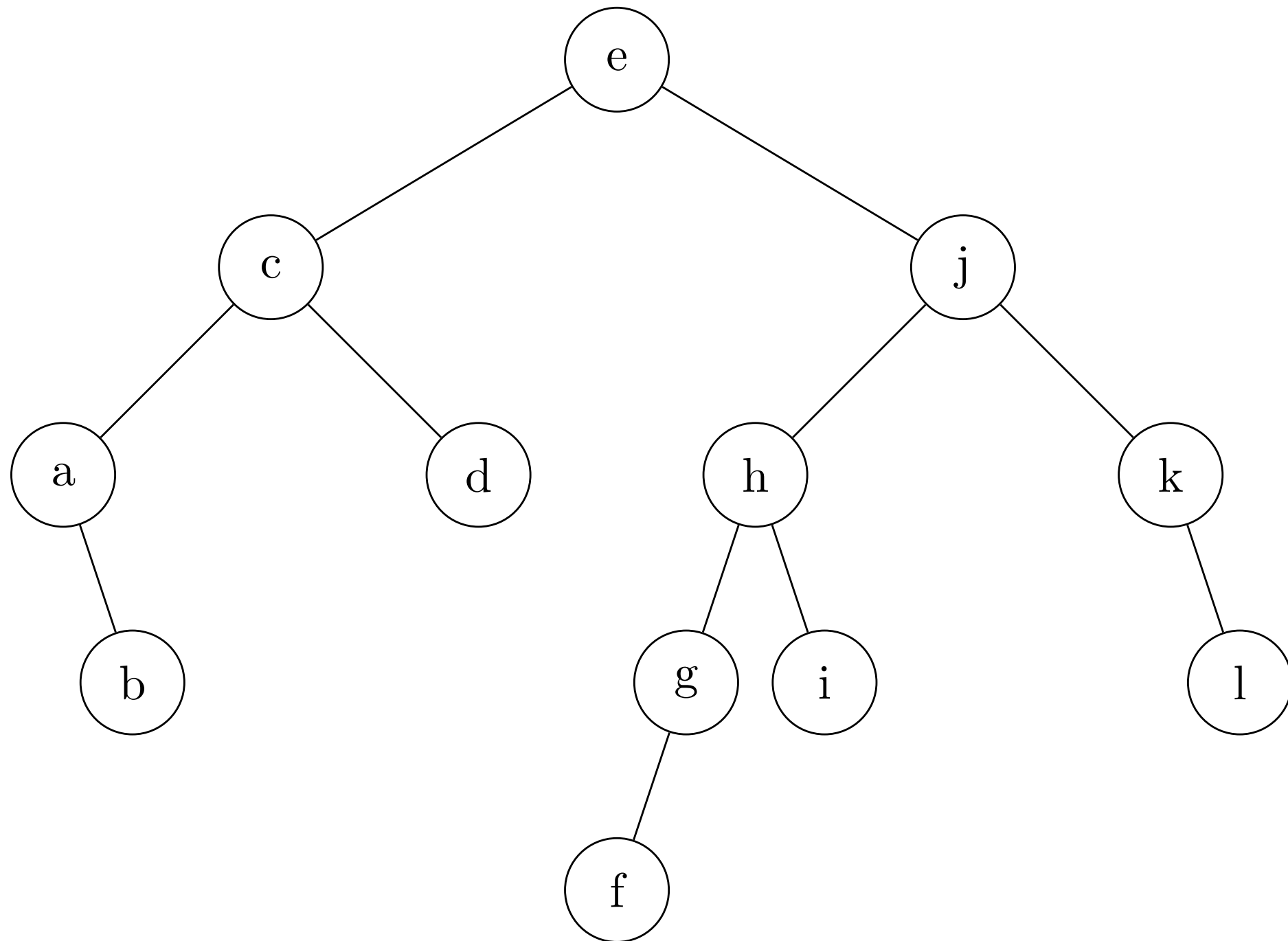


Figura: Remoção em Árvore AVL (b)

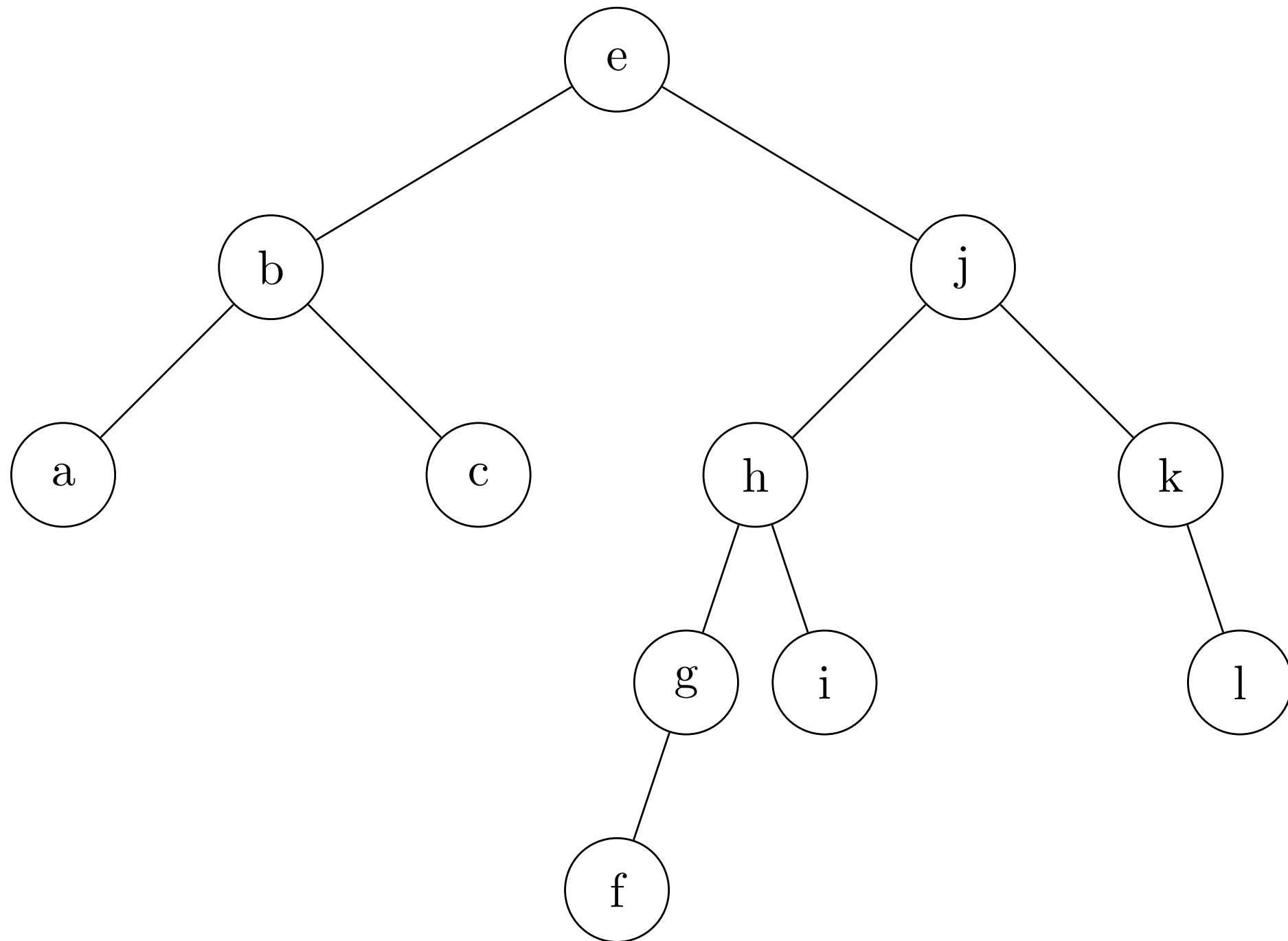
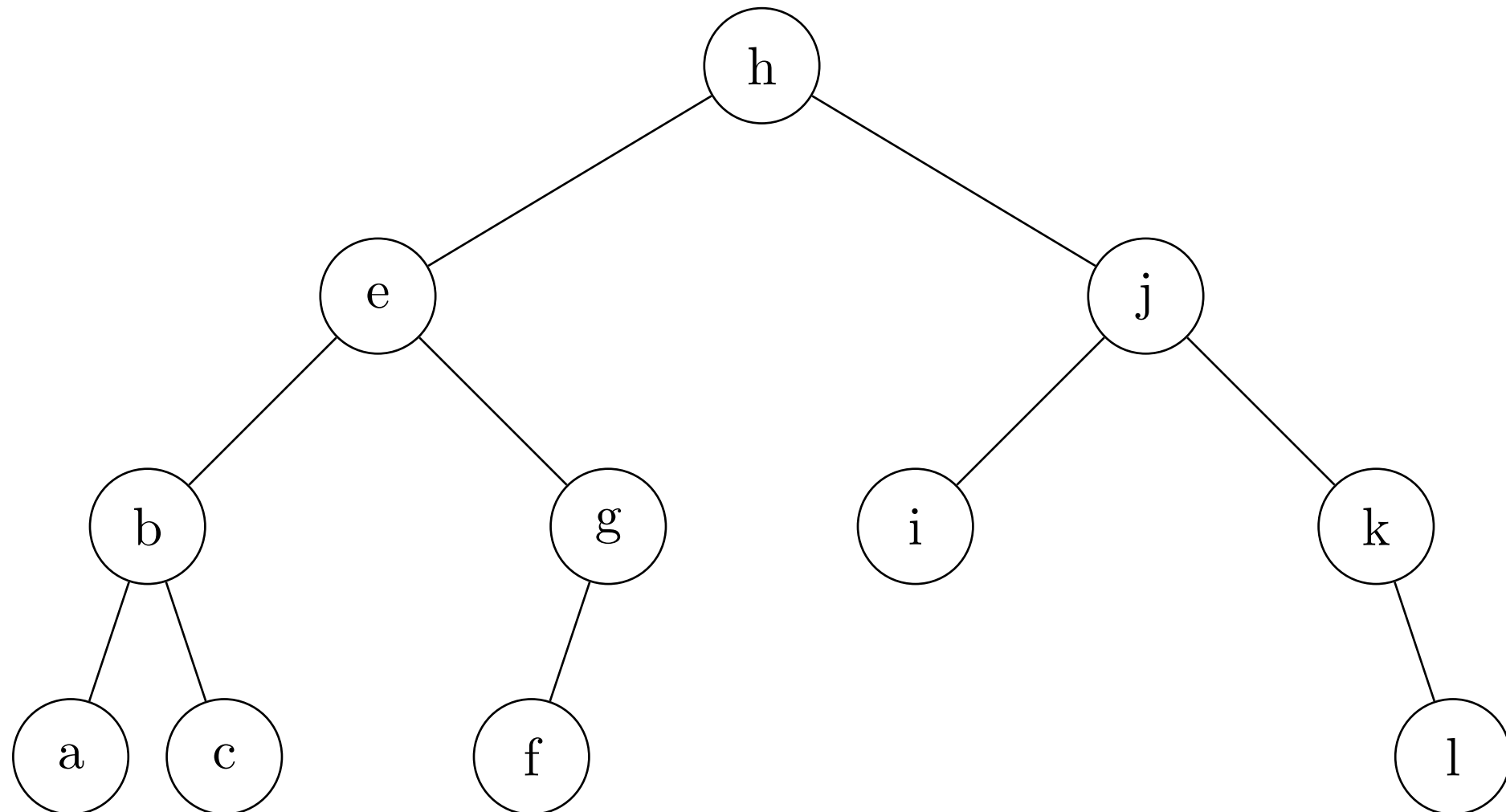


Figura: Remoção em Árvore AVL (c)



Referências

- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian.
Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora:
LTC. 2010